

Friendly Introduction to Deep Generative Models

Image synthesis version

Vitor N. Cabral de Oliveira

vnco@cin.ufpe.br

August 1, 2026

This notebook will serve as a comprehensive repository for my notes for an introductory course about Deep Generative Models, with a particular focus on image generation. The content will cover the following key topics:

- Style Transfer
- Autoencoders
- Variational Autoencoders
- Generative Adversarial Networks
- Diffusion Models

Future updates will introduce dedicated sections on energy-based models, flow-based models, and Boltzmann machines. To establish a rigorous framework, a preliminary section will review the statistical learning principles that serve as the agnostic mathematical foundation for generative modeling, using Hidden Markov Models as a classic introductory paradigm.

Contents

Roteiro. Este documento percorre a evolução dos modelos generativos profundos para síntese de imagens seguindo uma progressão narrativa em que cada abordagem endereça limitações da anterior.

Começamos com **Style Transfer**, um aquecimento que demonstra como CNNs separam e recombina conteúdo e estilo usando representações hierárquicas. Isso nos leva aos **Autoencoders**, que aprendem representações latentes compactas de forma não supervisionada. Mas autoencoders clássicos são determinísticos — não geram novos dados. Os **VAEs** introduzem um espaço latente probabilístico que permite amostragem, porém as amostras tendem a ser borradas. As **GANs** surgem como paradigma adversarial para qualidade foto-realista, mas com treinamento instável. Os **Modelos de Difusão** oferecem qualidade similar com estabilidade, inspirando-se em processos termodinâmicos. Os **Rectified Flow Models** simplificam e aceleram a difusão aprendendo fluxos ODE diretos. Por fim, os **Modelos Baseados em Energia** oferecem uma perspectiva alternativa modelando densidades via funções de energia.

Style Transfer

Definição 0.1 (Style Transfer). *A transferência de estilo é uma técnica de visão computacional que utiliza redes neurais convolucionais (CNNs) para gerar uma nova imagem que combina o conteúdo de uma imagem com o estilo artístico de outra. Essa técnica foi proposta no artigo seminal A Neural Algorithm of Artistic Style por Leon Gatys et al. e faz uso de redes convolucionais pré-treinadas, como a ResNet50, para extrair características visuais de diferentes níveis de abstração.*

A metodologia se baseia na relação entre as múltiplas camadas de uma CNN, onde cada camada l produz um conjunto de mapas de características $F^l(I)$. Esses mapas capturam informações visuais em diferentes níveis de complexidade, desde bordas e texturas nas camadas iniciais até formas e objetos nas camadas mais profundas. Assim, a CNN pode ser vista como um extrator hierárquico de características, onde cada camada contribui para a representação da imagem I .

O processo de transferência de estilo depende de duas imagens de entrada:

1. **Imagem de conteúdo (C):** Contém a estrutura e os objetos principais que serão preservados na imagem gerada.
2. **Imagem de estilo (S):** Fornece as texturas, cores e padrões artísticos que serão transferidos para a imagem final.

A imagem gerada (G) é obtida por meio da otimização de uma

função de perda composta, que combina a representação do conteúdo e do estilo.

Representação do Conteúdo

O conteúdo de uma imagem está associado às características de alto nível, como formas e objetos, que são capturados principalmente pelas camadas mais profundas da CNN. Para representar o conteúdo, utilizamos os mapas de características $F^l(C)$ e $F^l(G)$ extraídos de uma camada específica l da rede.

A função de perda do conteúdo mede a diferença entre as representações da imagem de conteúdo e da imagem gerada na camada l . Essa diferença é calculada como o erro quadrático médio entre os mapas de características:

$$\mathcal{L}_{\text{conteúdo}}(C, G, l) = \frac{1}{2} \sum_{i,j} \left(F_{ij}^l(G) - F_{ij}^l(C) \right)^2 \quad (1)$$

Onde: - $F_{ij}^l(C)$ é o valor do mapa de características da imagem de conteúdo na posição (i, j) da camada l . - $F_{ij}^l(G)$ é o valor correspondente na imagem gerada.

Representação do Estilo

O estilo de uma imagem está associado às características de baixo e médio nível, como texturas, cores e padrões, que são capturados pelas correlações entre os mapas de características em **múltiplas camadas** L da CNN. Para representar o estilo, utilizamos a matriz de Gram G^l , que mede as correlações entre os mapas de características na camada l .

A função de perda do estilo é definida como a soma ponderada das diferenças entre as matrizes de Gram da imagem de estilo e da imagem gerada em várias camadas:

$$\mathcal{L}_{\text{estilo}}(S, G, L) = \sum_{l \in L} w_l \cdot \frac{1}{4N_l^2 M_l^2} \sum_{i,j} \left(G_{ij}^l(G) - G_{ij}^l(S) \right)^2 \quad (2)$$

Onde: - w_l é o peso associado à camada l , que controla sua contribuição para a perda total. - N_l é o número de mapas de características na camada l . - M_l é o tamanho espacial (altura $\times$ largura) dos mapas de características na camada l . - $G_{ij}^l(S)$ e $G_{ij}^l(G)$ são os elementos das matrizes de Gram para a imagem de estilo e a imagem gerada, respectivamente.

Otimização

A imagem gerada G é obtida minimizando uma função de perda total, que combina as perdas de conteúdo e estilo:

$$\mathcal{L}_{\text{total}}(C, S, G) = \alpha \cdot \mathcal{L}_{\text{conteúdo}}(C, G, l) + \beta \cdot \mathcal{L}_{\text{estilo}}(S, G, L) \quad (3)$$

Onde α e β são hiperparâmetros que controlam o equilíbrio entre a preservação do conteúdo e a transferência do estilo.

Por meio de técnicas de otimização, como o gradiente descendente, a imagem gerada é iterativamente ajustada até que a perda total seja minimizada, resultando em uma imagem que combina o conteúdo de C com o estilo de S .

O objetivo é minimizar $\mathcal{L}_{\text{total}}$ em relação à imagem gerada G . Isso é feito usando Gradient Descent:

$$G \leftarrow G - \eta \cdot \nabla_G \mathcal{L}_{\text{total}}$$

Onde η é a taxa de aprendizado.

Matrizes de Gram na Transferência de Estilo

As matrizes de Gram desempenham um papel crucial na captura do estilo de uma imagem durante o processo de transferência de estilo. Elas são usadas para medir as correlações entre os mapas de características em diferentes camadas de uma rede neural convolucional (CNN). Esta seção fornece uma explicação detalhada das matrizes de Gram, seu cálculo e sua importância na transferência de estilo.

Definição das Matrizes de Gram

Dado um conjunto de mapas de características F^l extraídos de uma camada específica l de uma CNN, a matriz de Gram G^l é uma matriz quadrada que captura os produtos internos entre os mapas de características vetorizados. Se os mapas de características na camada l têm dimensões $N_l \times M_l$ (onde N_l é o número de mapas de características e M_l é a dimensão espacial, ou seja, altura $\times$ largura), a matriz de Gram G^l é definida como:

$$G^l = F^l \cdot (F^l)^T$$

Onde:

- F^l é a matriz de mapas de características vetorizados com dimensões $N_l \times M_l$.
- $(F^l)^T$ é a transposta de F^l .

- A matriz de Gram resultante G^l tem dimensões $N_l \times N_l$.

Cada elemento G_{ij}^l da matriz de Gram representa o produto interno entre o i -ésimo e o j -ésimo mapa de características:

$$G_{ij}^l = \sum_{k=1}^{M_l} F_{ik}^l \cdot F_{jk}^l$$

Interpretação das Matrizes de Gram

A matriz de Gram codifica informações sobre as correlações entre os mapas de características em uma determinada camada. Essas correlações capturam a textura e o estilo de uma imagem, pois representam a frequência com que certas características (por exemplo, bordas, cores, padrões) aparecem juntas na imagem. Por exemplo:

- Valores altos na matriz de Gram indicam correlações fortes entre mapas de características específicos, o que corresponde a padrões ou texturas recorrentes na imagem.
- Valores baixos indicam correlações fracas ou inexistentes, sugerindo a ausência de tais padrões.

Ao comparar as matrizes de Gram da imagem de estilo e da imagem gerada, o algoritmo de transferência de estilo garante que a imagem gerada imite a textura e o estilo da imagem de estilo.

Matrizes de Gram na Função de Perda de Estilo

A função de perda de estilo na transferência de estilo é calculada usando as matrizes de Gram da imagem de estilo (S) e da imagem gerada (G). Para uma camada específica l , a perda de estilo é definida como o erro quadrático médio entre as matrizes de Gram das duas imagens:

$$\mathcal{L}_{\text{estilo}}^l(S, G) = \frac{1}{4N_l^2 M_l^2} \sum_{i,j} \left(G_{ij}^l(G) - G_{ij}^l(S) \right)^2$$

Onde:

- $G_{ij}^l(S)$ é o elemento da matriz de Gram para a imagem de estilo.
- $G_{ij}^l(G)$ é o elemento da matriz de Gram para a imagem gerada.
- N_l é o número de mapas de características na camada l .
- M_l é a dimensão espacial dos mapas de características na camada l .

A perda de estilo total é calculada como uma soma ponderada das perdas de estilo em várias camadas L :

$$\mathcal{L}_{\text{estilo}}(S, G, L) = \sum_{l \in L} w_l \cdot \mathcal{L}_{\text{estilo}}^l(S, G)$$

Onde w_l é o peso atribuído à camada l .

Exemplo 0.1 (Cálculo da Matriz de Gram). Considere um exemplo simplificado em que uma camada l tem $N_l = 3$ mapas de características, cada um com dimensão espacial de $M_l = 2 \times 2 = 4$. Os mapas de características podem ser representados como:

$$F^l = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 2 & 0 & 1 & 3 \\ 1 & 1 & 2 & 2 \end{bmatrix}$$

A matriz de Gram G^l é calculada como:

$$G^l = F^l \cdot (F^l)^T = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 2 & 0 & 1 & 3 \\ 1 & 1 & 2 & 2 \end{bmatrix} \cdot \begin{bmatrix} 1 & 2 & 1 \\ 2 & 0 & 1 \\ 3 & 1 & 2 \\ 4 & 3 & 2 \end{bmatrix} = \begin{bmatrix} 30 & 20 & 20 \\ 20 & 14 & 13 \\ 20 & 13 & 14 \end{bmatrix}$$

Cada elemento de G^l representa a correlação entre os mapas de características correspondentes.

Significância das Matrizes de Gram

As matrizes de Gram são essenciais para a transferência de estilo porque fornecem uma maneira de quantificar e comparar a textura e o estilo das imagens. Ao minimizar a diferença entre as matrizes de Gram da imagem de estilo e da imagem gerada, o algoritmo garante que a imagem gerada adote o estilo artístico da imagem de estilo enquanto preserva o conteúdo da imagem de conteúdo.

A separação entre conteúdo e estilo nos mostrou que redes profundas aprendem representações hierárquicas poderosas. Os Autoencoders dão o próximo passo: aprender, de forma não supervisionada, representações latentes compactas que capturam a estrutura essencial dos dados — comprimindo sem perder o que é importante, um pré-requisito para a geração.

Autoencoders

Autoencoders (AEs) são uma técnica de aprendizado não supervisionado que visa aprender representações compactas e eficientes

de dados sem a necessidade de rótulos. Eles consistem em duas redes neurais principais: o **encoder** (codificador) e o **decoder** (decodificador). O encoder transforma os dados de entrada em uma representação codificada, enquanto o decoder tenta reconstruir os dados originais a partir dessa representação. Matematicamente, um autoencoder é definido por:

$$E_\phi : \mathcal{X} \rightarrow \mathcal{Z} \quad \text{e} \quad D_\theta : \mathcal{Z} \rightarrow \mathcal{X} \quad (4)$$

Onde:

- E_ϕ é o encoder, parametrizado por ϕ .
- D_θ é o decoder, parametrizado por θ .
- $\mathcal{X}$ é o espaço dos dados de entrada (por exemplo, $\mathbb{R}^m$).
- $\mathcal{Z}$ é o espaço latente (ou espaço codificado), geralmente de dimensão menor que $\mathcal{X}$ (por exemplo, $\mathbb{R}^n$, com $n \leq m$).

O objetivo do autoencoder é aprender uma representação compacta $z \in \mathcal{Z}$ dos dados de entrada $x \in \mathcal{X}$, de modo que a reconstrução $\hat{x} = D_\theta(E_\phi(x))$ seja o mais próxima possível de x .

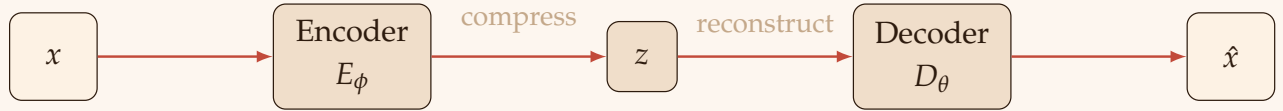


Figure 1: Exemplo clássico da arquitetura de um Autoencoder (AE).

Espaço Latente

Definição 0.2 (Espaço Latente). O espaço latente $\mathcal{Z}$ é uma representação de dimensionalidade reduzida dos dados de entrada. Para cada dado $x \in \mathcal{X}$, o encoder mapeia x para um vetor latente $z \in \mathcal{Z}$:

$$z = E_\phi(x) \quad (5)$$

Esse vetor latente z captura as características essenciais dos dados de entrada, eliminando redundâncias e ruídos. A dimensionalidade de $\mathcal{Z}$ é tipicamente menor que a de $\mathcal{X}$, o que permite ao autoencoder aprender uma representação compacta e eficiente.

Funcionamento do Autoencoder

A intuição por trás de um autoencoder pode ser resumida em três etapas principais:

1. **Entrada:** Um dado $x \in \mathbb{R}^D$ é fornecido como entrada.

2. **Encoder:** O encoder mapeia x para um vetor latente $z \in \mathbb{R}^d$, onde $d \ll D$:

$$z = E(x) = \sigma(W_e x + b_e) \quad (6)$$

Aqui, W_e e b_e são os pesos e vieses do encoder, respectivamente, e σ é uma função de ativação não linear (como ReLU ou sigmoide).

3. **Decoder** O decoder reconstrói $\hat{x} \in \mathbb{R}^D$ a partir do vetor latente z :

$$\hat{x} = D(z) = \sigma(W_d z + b_d) \quad (7)$$

Onde W_d e b_d são os pesos e vieses do decoder.

Função de Perda

O objetivo do autoencoder é minimizar o erro de reconstrução, ou seja, a diferença entre a entrada original x e a reconstrução $\hat{x}$. A função de perda mais comum é o erro quadrático médio (MSE), definido como:

$$\mathcal{L}_{AE} = \mathbb{E}_{x \sim p_{\text{data}}} [\|x - \hat{x}\|^2] \quad (8)$$

Onde:

- $\mathbb{E}_{x \sim p_{\text{data}}}$ denota o valor esperado em relação à distribuição dos dados de entrada.
- $\|x - \hat{x}\|^2$ é a norma euclidiana ao quadrado da diferença entre x e $\hat{x}$.

Ao minimizar essa função de perda, o autoencoder aprende a reconstruir os dados de entrada com a maior precisão possível, ao mesmo tempo em que compacta a informação no espaço latente.

Variações de Autoencoders

Além do autoencoder básico, existem várias variações que ampliam suas capacidades:

- **Denoising Autoencoder:** Treinado para reconstruir dados a partir de entradas corrompidas por ruído.
- **Sparse Autoencoder:** Introduce uma penalidade de esparsidade no espaço latente, incentivando a ativação de apenas algumas unidades latentes.
- **Variational Autoencoder (VAE):** Modela a distribuição dos dados no espaço latente, permitindo a geração de novos dados.

- Convolutional Autoencoder: Utiliza camadas convolucionais no encoder e decoder, sendo especialmente útil para dados imagéticos.

Autoencoders clássicos aprendem um mapeamento determinístico $x \rightarrow z \rightarrow \hat{x}$, excelente para compressão mas insuficiente para geração: não há estrutura probabilística que permita amostrar novos z realistas. Os **Variational Autoencoders** preenchem exatamente essa lacuna, modelando o espaço latente probabilisticamente — cada x é mapeado para uma distribuição $q_\phi(z|x)$ em vez de um único vetor z .

Variational Autoencoders (VAEs)

Os Variational Autoencoders (VAEs) são uma extensão dos autoencoders tradicionais que incorporam conceitos de inferência probabilística. Enquanto autoencoders comuns aprendem uma representação determinística no espaço latente, os VAEs modelam a distribuição de probabilidade dos dados no espaço latente, permitindo a geração de novos dados a partir dessa distribuição.

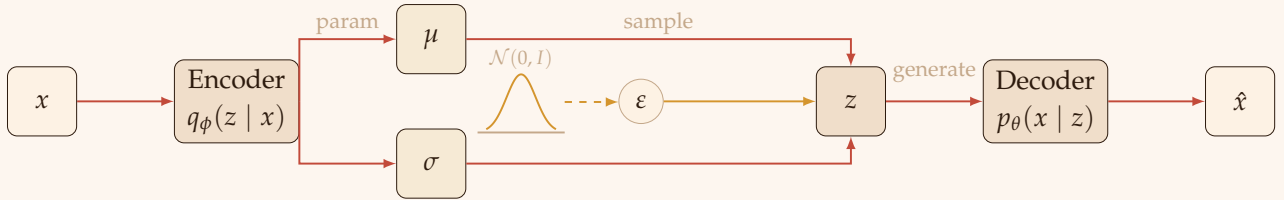


Figure 2: Arquitetura de um Variational Autoencoder (VAE) detalhando o reparameterization trick em formato expandido com amostragem exógena de uma distribuição $\mathcal{N}(0, I)$.

Fundamentos Probabilísticos

Ao contrário dos autoencoders tradicionais, que mapeiam diretamente uma entrada x para um vetor latente z , os VAEs modelam a distribuição de probabilidade $p(z|x)$. No entanto, como essa distribuição é intratável, os VAEs utilizam uma abordagem variacional para aproximá-la. Especificamente, eles introduzem uma distribuição aproximada $q_\phi(z|x)$, parametrizada por ϕ , que é aprendida durante o treinamento.

O objetivo do VAE é maximizar a ELBO, que é uma aproximação do logaritmo da verossimilhança dos dados. A ELBO é dada por:

$$\text{ELBO} = \mathbb{E}_{q_\phi(z|x)} [\log p_\theta(x|z)] - D_{\text{KL}}(q_\phi(z|x) || p(z)) \quad (9)$$

Onde:

- $\mathbb{E}_{q_\phi(z|x)} [\log p_\theta(x|z)]$ é o termo de reconstrução, que mede quão bem o decoder consegue reconstruir x a partir de z .

- $D_{\text{KL}}(q_\phi(z|x) || p(z))$ é a divergência de Kullback-Leibler (KL) entre a distribuição aproximada $q_\phi(z|x)$ e uma distribuição pré-definida $p(z)$ (geralmente uma distribuição normal padrão $\mathcal{N}(0, I)$).

Encoder e Decoder Probabilísticos

No VAE, tanto o encoder quanto o decoder são modelados probabilisticamente:

1. **Encoder:** O encoder mapeia a entrada x para os parâmetros de uma distribuição no espaço latente, geralmente uma distribuição normal multivariada:

$$q_\phi(z|x) = \mathcal{N}(z; \mu_\phi(x), \sigma_\phi^2(x)) \quad (10)$$

Onde $\mu_\phi(x)$ e $\sigma_\phi^2(x)$ são a média e a variância da distribuição, respectivamente, aprendidas pelo encoder.

2. **Decoder:** O decoder mapeia o vetor latente z para os parâmetros de uma distribuição nos dados de saída. Para dados contínuos, isso geralmente é uma distribuição normal:

$$p_\theta(x|z) = \mathcal{N}(x; \mu_\theta(z), \sigma_\theta^2(z)) \quad (11)$$

Para dados binários, uma distribuição Bernoulli pode ser usada.

Reparametrização

Um desafio no treinamento de VAEs é a amostragem de z a partir de $q_\phi(z|x)$, que é uma operação estocástica e não diferenciável. Para contornar esse problema, os VAEs utilizam o **truque da reparametrização**: em vez de amostrar diretamente de $q_\phi(z|x)$, eles amostram de uma distribuição base $\epsilon \sim \mathcal{N}(0, I)$ e transformam a amostra da seguinte forma:

$$z = \mu_\phi(x) + \sigma_\phi(x) \cdot \epsilon \quad (12)$$

Essa transformação torna o processo diferenciável, permitindo o uso de gradientes descendentes para otimizar os parâmetros do modelo.

Função de Perda

A função de perda do VAE é o negativo da ELBO, que consiste em dois termos:

1. Termo de reconstrução: Mede a qualidade da reconstrução dos dados de entrada.

2. Termo de regularização: A divergência KL, que força a distribuição aproximada $q_\phi(z|x)$ a se aproximar da distribuição pré-definida $p(z)$.

Matematicamente, a função de perda é dada por:

$$\mathcal{L}_{\text{VAE}} = -\mathbb{E}_{q_\phi(z|x)} [\log p_\theta(x|z)] + D_{\text{KL}}(q_\phi(z|x) \| p(z)) \quad (13)$$

Exemplo de Arquitetura

Um VAE típico consiste em:

1. Encoder: Uma rede neural que mapeia a entrada x para os parâmetros $\mu_\phi(x)$ e $\sigma_\phi(x)$ da distribuição latente.
2. Reparametrização: Amostra z usando o truque da reparametrização.
3. Decoder: Uma rede neural que mapeia z para os parâmetros da distribuição de saída $p_\theta(x|z)$.

Vantagens e Desvantagens

Vantagens

- Permitem a geração de novos dados a partir de uma distribuição aprendida.
- O espaço latente é contínuo, o que facilita a interpolação e a manipulação de dados.
- São baseados em fundamentos probabilísticos sólidos.

Desvantagens

- As amostras geradas podem ser menos nítidas ou realistas em comparação com outras técnicas, como GANs (Generative Adversarial Networks).
- O treinamento pode ser mais complexo devido à necessidade de otimizar a ELBO.

VAEs oferecem um espaço latente contínuo e bem-estruturado, mas as amostras geradas tendem a ser borradas — a perda de reconstrução penaliza desvios pixel-a-pixel, incentivando médias sobre modos plausíveis. As **Generative Adversarial Networks** introduzem um paradigma radicalmente diferente: em vez de minimizar uma distância explícita, um discriminador aprende a distinguir amostras reais de falsas, forçando o gerador a produzir amostras indistinguíveis das reais.

Generative Adversarial Networks (GANs)

As Generative Adversarial Networks (GANs) são uma classe de modelos de aprendizado profundo introduzidos por Ian Goodfellow e colaboradores em 2014. Elas são amplamente utilizadas para geração de dados, como imagens, áudio e texto. A ideia central das GANs é treinar dois modelos neurais simultaneamente: um **gerador** (generator) e um **discriminador** (discriminator), que competem entre si em um jogo de soma zero. Essa competição leva à geração de dados realistas que são indistinguíveis dos dados reais.

Definição 0.3 (Jogo Minimax das GANs). *Uma GAN consiste em dois componentes principais:*

1. *Gerador (G): Um modelo que gera dados a partir de um vetor de ruído aleatório z , geralmente amostrado de uma distribuição simples, como uma distribuição normal ou uniforme. O objetivo do gerador é produzir dados que sejam indistinguíveis dos dados reais.*
2. *Discriminador (D): Um modelo que tenta distinguir entre dados reais (amostrados da distribuição real $p_{data}(x)$) e dados falsos (gerados pelo gerador). O discriminador é treinado para maximizar a probabilidade de classificar corretamente os dados como reais ou falsos.*

O treinamento das GANs é formulado como um jogo minimax entre o gerador e o discriminador, onde o gerador tenta enganar o discriminador, e o discriminador tenta detectar as amostras falsas. Matematicamente, o objetivo pode ser expresso como:

$$\min_G \max_D V(D, G) = \mathbb{E}_{x \sim p_{data}(x)} [\log D(x)] + \mathbb{E}_{z \sim p_z(z)} [\log(1 - D(G(z)))] \quad (14)$$

Onde:

- $D(x)$ é a probabilidade estimada pelo discriminador de que x seja real.
- $G(z)$ é a amostra gerada pelo gerador a partir do vetor de ruído z .
- $p_{data}(x)$ é a distribuição dos dados reais.
- $p_z(z)$ é a distribuição do vetor de ruído (geralmente $\mathcal{N}(0, I)$ ou $\text{Uniform}(-1, 1)$).

Funcionamento do Treinamento

O treinamento de uma GAN envolve as seguintes etapas iterativas:

1. Treinamento do Discriminador: Fixa-se o gerador e atualiza-se o discriminador para maximizar a função de custo $V(D, G)$. Isso envolve:
 - Classificar corretamente os dados reais como reais.
 - Classificar corretamente os dados falsos (gerados pelo gerador) como falsos.
2. Treinamento do Gerador: Fixa-se o discriminador e atualiza-se o gerador para minimizar a função de custo $V(D, G)$. Isso envolve:
 - Gerar dados que sejam classificados como reais pelo discriminador.

Esse processo é repetido até que o gerador produza dados realistas e o discriminador não consiga distinguir entre dados reais e falsos.

Funções de Perda

A função de perda das GANs pode ser interpretada como uma medida de divergência entre a distribuição real $p_{\text{data}}(x)$ e a distribuição gerada $p_g(x)$. No entanto, o treinamento das GANs é conhecido por ser instável, o que levou ao desenvolvimento de várias variações e funções de perda alternativas, como:

- GAN Padrão: Usa a função de perda binária cruzada (binary cross-entropy).
- Wasserstein GAN (WGAN): Utiliza a distância de Wasserstein para melhorar a estabilidade do treinamento.
- Least Squares GAN (LSGAN): Substitui a função de perda binária cruzada por uma função de perda quadrática.

Comparação com Outras Técnicas

As GANs são frequentemente comparadas com outras técnicas de geração de dados, como Variational Autoencoders (VAEs):

- GANs: Produzem amostras de alta qualidade, mas são mais difíceis de treinar e avaliar.
- VAEs: São mais estáveis e fornecem uma representação probabilística do espaço latente, mas as amostras geradas podem ser menos nítidas.

Interpretação de GANs via Teoria de Jogos

O treinamento adversarial das GANs, apresentado na Equação 4.1, pode ser formalizado como um jogo entre dois jogadores — o gerador G e o discriminador D . Esta subseção aprofunda essa interpretação utilizando conceitos da teoria de jogos, conforme o artigo original de Goodfellow et al. (2014).

Funções de Perda Individuais

Embora a equação minimax represente o objetivo geral do jogo, o artigo original define as funções de perda individuais do discriminador e do gerador como:

$$J^{(D)}(\theta_d, \theta_g) = -\frac{1}{2}\mathbb{E}_{x \sim p_{\text{data}}(x)} [\log D(x)] - \frac{1}{2}\mathbb{E}_{z \sim p_z(z)} [\log(1 - D(G(z)))] \quad (15)$$

$$J^{(G)}(\theta_d, \theta_g) = -J^{(D)}(\theta_d, \theta_g) \quad (16)$$

No entanto, para lidar com problemas de saturação e gradientes fracos no início do treinamento, o artigo original também propõe uma **alternativa** à função de perda do gerador, conhecida como **heurística de saturação não decrescente**:

$$J^{(G)}(\theta_d, \theta_g) = -\frac{1}{2}\mathbb{E}_{z \sim p_z(z)} [\log(D(G(z)))] \quad (17)$$

Essa modificação fornece gradientes mais fortes para o gerador quando o discriminador rejeita facilmente as amostras geradas.

Estratégias dos Jogadores

Cada jogador (gerador e discriminador) tem uma estratégia que consiste em ajustar seus parâmetros (θ_g e θ_d , respectivamente) para otimizar sua função objetivo:

1. **Discriminador (D)**: Escolhe seus parâmetros θ_d para maximizar $J^{(D)}(\theta_d, \theta_g)$. Isso envolve classificar corretamente os dados reais como reais e os dados falsos como falsos. O discriminador é atualizado usando gradiente ascendente em sua função de perda.
2. **Gerador (G)**: Escolhe seus parâmetros θ_g para minimizar $J^{(G)}(\theta_d, \theta_g)$, seja usando a formulação original (minimizando a divergência de Jensen-Shannon¹) ou a heurística de saturação não decrescente. O gerador é atualizado usando algoritmos de otimização baseados em gradiente em sua função de perda.

¹ A divergência de Jensen-Shannon é uma derivação da divergência de Kullback-Leibler, com diferenças sutis como o resultado ser sempre um valor finito. Sua raiz quadrada é chamada de Distância de Jensen-Shannon: a similaridade das distribuições é maior quando a distância for mais próxima à zero.

O treinamento das GANs pode ser visto como um processo iterativo em que os jogadores ajustam suas estratégias em resposta às ações do oponente. O artigo original sugere atualizar o discriminador k vezes para cada atualização do gerador.

Teorema 0.1 (Equilíbrio de Nash). *Para entender o conceito de Equilíbrio de Nash em GANs, é útil defini-lo formalmente no contexto da teoria de jogos. Considere um jogo com dois jogadores, onde:*

- G é o gerador, com espaço de estratégias S_G (o conjunto de todas as possíveis distribuições que o gerador pode produzir).
- D é o discriminador, com espaço de estratégias S_D (o conjunto de todas as possíveis funções discriminadoras).
- $V(G, D)$ é a função de valor (ou payoff) que define o resultado do jogo para cada combinação de estratégias.

Definição: Um par de estratégias (G^*, D^*) é um **Equilíbrio de Nash** se, e somente se, as seguintes condições forem satisfeitas:

1. Melhor resposta do discriminador: Para qualquer estratégia alternativa do discriminador $D \in S_D$, temos:

$$V(G^*, D^*) \geq V(G^*, D) \quad (18)$$

Isso significa que, dado que o gerador está jogando a estratégia G^* , o discriminador não pode obter um resultado melhor do que jogar D^* . Em outras palavras, D^* é a melhor resposta do discriminador à estratégia G^* .

2. Melhor resposta do gerador: Para qualquer estratégia alternativa do gerador $G \in S_G$, temos:

$$V(G^*, D^*) \leq V(G, D^*) \quad (19)$$

Isso significa que, dado que o discriminador está jogando a estratégia D^* , o gerador não pode obter um resultado melhor do que jogar G^* . Em outras palavras, G^* é a melhor resposta do gerador à estratégia D^* .

Em termos matemáticos mais compactos:

Um par de estratégias (G^*, D^*) é um Equilíbrio de Nash se:

$$V(G^*, D^*) = \max_{D \in S_D} V(G^*, D) = \min_{G \in S_G} V(G, D^*) \quad (20)$$

No equilíbrio de Nash, nenhum jogador tem incentivo para mudar sua estratégia unilateralmente. Qualquer desvio da estratégia de equilíbrio resultará em um resultado pior para o jogador que se desviou, assumindo que o outro jogador permaneça com sua estratégia de equilíbrio.

No contexto das GANs, o equilíbrio de Nash global idealmente ocorre quando:

- O gerador G^* produz amostras da distribuição real de dados ($p_g(x) = p_{data}(x)$).
- O discriminador D^* atribui uma probabilidade de $1/2$ para todas as entradas ($D^*(x) = 1/2$ para todo x), indicando que ele não pode distinguir entre amostras reais e geradas.

Importante:

- A existência de um equilíbrio de Nash é garantida em jogos finitos (com um número finito de jogadores e estratégias) pelo Teorema de Nash. No entanto, GANs operam em espaços de estratégias contínuos e de alta dimensão, tornando a análise mais complexa.
- A definição acima refere-se ao **equilíbrio de Nash puro**. Existem também **equilíbrios de Nash mistos**, onde os jogadores escolhem suas estratégias probabilisticamente.

Teorema 0.2 (Conexão com Jensen-Shannon). Quando o discriminador é ótimo, *minimizar a função de valor $V(G, D)$ é equivalente a minimizar a divergência de Jensen-Shannon (JSD) entre p_{data} e p_g , conforme a seguinte relação:*

$$V(G, D_G^*) = 2 \cdot D_{JS}(p_{data} || p_g) - \log 4 \quad (21)$$

Isso significa que, sob condições ideais, o treinamento de GANs visa minimizar a JSD, uma medida simétrica de similaridade entre distribuições.

Convergência do Treinamento

Embora o artigo original apresente uma prova teórica de convergência para um algoritmo idealizado, ele também reconhece as dificuldades práticas, como:

1. **Capacidade Limitada dos Modelos:** Redes neurais têm capacidade finita.
2. **Otimização Imperfeita:** O discriminador nem sempre atinge seu ótimo na prática.
3. **Aproximações da Função de Perda:** A heurística de saturação não decrescente, embora útil, não possui a mesma base teórica que a minimização da JSD.

Esses desafios levam a problemas bem conhecidos no treinamento de GANs:

- **Instabilidade:** A natureza antagônica do jogo pode causar oscilações e dificultar a convergência.

- **Colapso de Modo:** O gerador pode aprender a produzir apenas uma variedade limitada de amostras, perdendo a diversidade da distribuição real.

Exemplo 0.2 (GAN Unidimensional). Considere um caso simplificado para ilustrar o conceito de equilíbrio de Nash:

- Espaço de dados unidimensional: $x \in \mathbb{R}$.
- Distribuição real: $p_{\text{data}}(x) = \mathcal{N}(0, 1)$ (Gaussiana padrão).
- Gerador: $G(z) = \theta_g z$, onde $z \sim \mathcal{N}(0, 1)$.
- Discriminador: $D(x) = \sigma(\theta_d x)$, onde σ é a função sigmoide.

Neste cenário, o equilíbrio de Nash ocorre quando $\theta_g = 1$ e $\theta_d = 0$. Isso significa que:

- O gerador replica perfeitamente a distribuição real.
- O discriminador se torna incapaz de distinguir entre dados reais e falsos, atribuindo uma probabilidade de 0,5 para ambos.

Demonstração. Para um gerador fixo, o discriminador ótimo é:

$$D_G^*(x) = \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_g(x)} = \frac{\mathcal{N}(x; 0, 1)}{\mathcal{N}(x; 0, 1) + \mathcal{N}(x; 0, \theta_g^2)} \quad (22)$$

No equilíbrio de Nash, $p_g(x) = p_{\text{data}}(x)$, o que implica $\theta_g = 1$. Nesse caso, $D_G^*(x) = 1/2$.

Para encontrar o θ_d ótimo, pode-se derivar a função de perda do discriminador em relação a θ_d . Substituindo $D(x)$ pela expressão do discriminador ótimo $D_G^*(x)$ e considerando o equilíbrio de Nash ($\theta_g = 1$), onde $p_g(x) = p_{\text{data}}(x)$, a derivada se simplifica e se torna zero quando $\theta_d = 0$. Isso indica que, no equilíbrio, o discriminador se torna uma função constante, atribuindo a probabilidade de 0,5 para qualquer entrada.

Este exemplo é extremamente simplificado e serve apenas para fins ilustrativos. Em casos reais, encontrar o equilíbrio de Nash analiticamente é inviável, e o treinamento de GANs depende de métodos numéricos para aproximá-lo.

Apesar da qualidade impressionante das amostras, as GANs são notoriamente instáveis durante o treinamento — colapso de modo e oscilações são desafios constantes. Os **Modelos de Difusão** oferecem uma alternativa que alcança qualidade similar, ou superior, com treinamento estável, inspirada em processos termodinâmicos de difusão e reversão.

Modelos de Difusão

Os Modelos de Difusão (Diffusion Models) são uma classe de modelos generativos que se baseiam em um processo de difusão para transformar dados reais em ruído e, em seguida, aprender a reverter esse processo para gerar novos dados. Eles têm se destacado por sua capacidade de produzir amostras de alta qualidade e por sua estabilidade de treinamento, superando desafios comuns em outras abordagens, como GANs (Generative Adversarial Networks) e VAEs (Variational Autoencoders).

Visão Geral do Processo de Difusão

O conceito central dos Modelos de Difusão é simular um processo de difusão, que gradualmente adiciona ruído aos dados reais até que eles se tornem indistinguíveis de ruído puro. Em seguida, o modelo aprende a reverter esse processo, transformando o ruído de volta em dados realistas. Esse processo é dividido em duas fases principais:

1. **Fase de Difusão (Forward Process):** Adiciona ruído aos dados reais em várias etapas, seguindo um esquema pré-definido.
2. **Fase de Reversão (Reverse Process):** Aprende a remover o ruído e reconstruir os dados originais.

Fase de Difusão (Forward Process)

Definição 0.4 (Processo de Difusão). *Na fase de difusão, os dados reais $\mathbf{x}_0 \sim q(\mathbf{x})$ são gradualmente corrompidos por ruído gaussiano em T etapas. Em cada etapa t , o dado $\mathbf{x}_t$ é obtido a partir de $\mathbf{x}_{t-1}$ pela adição de ruído:*

$$q(\mathbf{x}_t | \mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x}_t; \sqrt{1 - \beta_t} \mathbf{x}_{t-1}, \beta_t \mathbf{I}) \quad (23)$$

Onde:

- β_t é um parâmetro que controla a quantidade de ruído adicionado na etapa t , geralmente definido por uma *schedule* de variâncias.
- $\mathbf{x}_{t-1}$ é a amostra da etapa anterior.
- $\mathbf{I}$ é a matriz identidade.

Definindo $\alpha_t = 1 - \beta_t$ e $\bar{\alpha}_t = \prod_{s=1}^t \alpha_s$, podemos obter diretamente $\mathbf{x}_t$ a partir de $\mathbf{x}_0$ em uma única etapa:

$$q(\mathbf{x}_t | \mathbf{x}_0) = \mathcal{N}(\mathbf{x}_t; \sqrt{\bar{\alpha}_t} \mathbf{x}_0, (1 - \bar{\alpha}_t) \mathbf{I}) \quad (24)$$

Isso significa que podemos amostrar $\mathbf{x}_t$ da seguinte forma:

$$\mathbf{x}_t = \sqrt{\tilde{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \tilde{\alpha}_t} \boldsymbol{\epsilon}, \quad \text{onde } \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}) \quad (25)$$

No final do processo de difusão ($t = T$), o dado $\mathbf{x}_T$ é essencialmente ruído gaussiano puro, sem qualquer informação discernível sobre $\mathbf{x}_0$, dado que a schedule de variâncias β_t é bem definida.

Fase de Reversão (Reverse Process)

Definição 0.5 (Processo de Reversão). A fase de reversão consiste em aprender a remover o ruído e reconstruir os dados originais. Isso é feito por meio de um modelo neural parametrizado por θ que aprende a prever a média e a variância da distribuição gaussiana que reverte o processo de difusão em cada etapa.

O processo de reversão é modelado como uma cadeia de Markov com transições gaussianas aprendidas:

$$p_\theta(\mathbf{x}_{t-1} | \mathbf{x}_t) = \mathcal{N}(\mathbf{x}_{t-1}; \boldsymbol{\mu}_\theta(\mathbf{x}_t, t), \boldsymbol{\Sigma}_\theta(\mathbf{x}_t, t)) \quad (26)$$

Onde:

- $\boldsymbol{\mu}_\theta(\mathbf{x}_t, t)$ é a média prevista pelo modelo neural.
- $\boldsymbol{\Sigma}_\theta(\mathbf{x}_t, t)$ é a variância prevista pelo modelo neural. Na prática, muitas vezes é fixada como $\sigma_t^2 \mathbf{I}$, onde σ_t^2 pode ser igual a β_t ou $\frac{1-\tilde{\alpha}_t-1}{1-\tilde{\alpha}_t} \beta_t$.

Em vez de prever diretamente $\mathbf{x}_{t-1}$, o modelo geralmente é treinado para prever o ruído $\boldsymbol{\epsilon}$ adicionado em cada etapa. Assim, a média $\boldsymbol{\mu}_\theta$ pode ser reparametrizada como:

$$\boldsymbol{\mu}_\theta(\mathbf{x}_t, t) = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{\beta_t}{\sqrt{1 - \tilde{\alpha}_t}} \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t) \right) \quad (27)$$

Portanto, o processo de amostragem para gerar $\mathbf{x}_{t-1}$ a partir de $\mathbf{x}_t$ é:

$$\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{\beta_t}{\sqrt{1 - \tilde{\alpha}_t}} \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t) \right) + \sigma_t \mathbf{z}, \quad \text{onde } \mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}) \quad (28)$$

Função de Perda

O treinamento dos Modelos de Difusão é baseado na maximização da verossimilhança dos dados de treinamento. No entanto, a verossimilhança exata é intratável. Em vez disso, os modelos de difusão são treinados otimizando um limite inferior variacional (ELBO) na log-verossimilhança negativa.

Uma simplificação comum da função de perda é:

$$\mathcal{L}(\theta) = \mathbb{E}_{t, \mathbf{x}_0, \boldsymbol{\epsilon}} \left[\left\| \boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\sqrt{\tilde{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \tilde{\alpha}_t} \boldsymbol{\epsilon}, t) \right\|^2 \right] \quad (29)$$

Onde:

- t é a etapa de difusão, amostrada uniformemente entre 1 e T .
- $\mathbf{x}_0$ é o dado real.
- $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ é o ruído adicionado durante a fase de difusão.
- $\epsilon_\theta(\mathbf{x}_t, t)$ é o ruído previsto pelo modelo, onde $\mathbf{x}_t = \sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon$.

Essa função de perda simplesmente compara o ruído real adicionado durante o processo de difusão com o ruído previsto pelo modelo.

Vantagens dos Modelos de Difusão

Os Modelos de Difusão apresentam várias vantagens em relação a outras técnicas generativas:

- **Estabilidade de Treinamento:** Ao contrário das GANs, que podem sofrer com instabilidade e colapso de moda, os Modelos de Difusão são mais estáveis e previsíveis.
- **Qualidade das Amostras:** Eles são capazes de gerar amostras de alta qualidade, especialmente em tarefas como geração de imagens.
- **Flexibilidade:** Podem ser aplicados a diferentes tipos de dados, como imagens, áudio e vídeos.
- **Fundamentação Teórica:** Baseiam-se em um processo bem definido de difusão e reversão, o que facilita a análise e a interpretação.

Comparação com Outras Técnicas

Em comparação com outras abordagens generativas, como GANs e VAEs, os Modelos de Difusão oferecem:

- **Melhor Qualidade:** As amostras geradas tendem a ser mais nítidas e realistas.
- **Maior Estabilidade:** O treinamento é menos propenso a problemas como colapso de moda.
- **Custo Computacional:** O treinamento e a geração podem ser mais lentos devido ao grande número de etapas de difusão e reversão.

Exemplo de Arquitetura: DDPM e U-Net

Um exemplo popular de arquitetura para Modelos de Difusão é o Denoising Diffusion Probabilistic Models (DDPM), que utiliza uma rede neural U-Net para prever o ruído $\epsilon_\theta(\mathbf{x}_t, t)$ em cada etapa de reversão. A U-Net é eficaz para capturar informações em múltiplas escalas, o que é crucial para a qualidade das amostras geradas.

A arquitetura U-Net consiste em um caminho de contração (encoder) que captura o contexto e um caminho de expansão (decoder) que permite a localização precisa. A U-Net também possui conexões residuais entre o encoder e o decoder, o que ajuda a preservar os detalhes finos durante o processo de geração.

Stable Diffusion

O Stable Diffusion é uma variação dos Modelos de Difusão que opera no **espaço latente** de um autoencoder variacional (VAE) pré-treinado, em vez de operar diretamente no espaço de pixels. Isso traz algumas vantagens:

1. **Eficiência Computacional:** Trabalhar em um espaço latente de menor dimensão reduz significativamente o custo computacional do processo de difusão e reversão.
2. **Velocidade de Amostragem:** A geração de amostras se torna mais rápida, pois o processo de reversão ocorre em um espaço de menor dimensão.
3. **Qualidade Aprimorada:** O modelo pode se concentrar em capturar a semântica e a estrutura de alto nível dos dados, enquanto o VAE cuida dos detalhes de baixo nível.

Funcionamento do Stable Diffusion

1. **Treinamento do VAE:** Um VAE é treinado para comprimir imagens em um espaço latente de menor dimensão. O encoder do VAE mapeia uma imagem $\mathbf{x}$ para uma representação latente $\mathbf{z} = E(\mathbf{x})$, e o decoder reconstrói a imagem a partir da representação latente: $\mathbf{x} \approx D(\mathbf{z})$.
2. **Difusão no Espaço Latente:** O processo de difusão é aplicado às representações latentes $\mathbf{z}$ em vez das imagens originais $\mathbf{x}$. Isso significa que $\mathbf{z}_t$ é obtido a partir de $\mathbf{z}_0$ (que é a representação latente de $\mathbf{x}_0$) adicionando ruído de acordo com a *schedule* de variâncias.
3. **Modelo de Difusão Condicionado:** Um modelo de difusão, geralmente uma U-Net, é treinado para reverter o processo de difusão

no espaço latente. Esse modelo pode ser condicionado a informações adicionais, como texto (para geração de imagem a partir de texto) ou máscaras de segmentação (para edição de imagens).

4. **Geração de Amostras:** Para gerar uma nova amostra, o processo de reversão é executado no espaço latente, começando com ruído gaussiano puro $\mathbf{z}_T$. Em seguida, o decoder do VAE é usado para mapear a representação latente gerada $\mathbf{z}_0$ de volta para o espaço de pixels, produzindo a imagem final $\mathbf{x} = D(\mathbf{z}_0)$.

Arquitetura do Stable Diffusion

O Stable Diffusion geralmente usa uma arquitetura U-Net para o modelo de difusão no espaço latente. Além disso, ele pode incorporar mecanismos de atenção cruzada (cross-attention) para condicionar a geração a entradas de texto. Isso permite que o modelo gere imagens que correspondam a descrições textuais específicas.

Em resumo, o Stable Diffusion é uma abordagem eficiente e poderosa para a geração de imagens de alta qualidade, combinando a força dos Modelos de Difusão com a eficiência de trabalhar em um espaço latente aprendido por um VAE. Modelos de Difusão produzem resultados excelentes, mas seu processo de amostragem é lento — são necessárias centenas ou milhares de etapas para gerar uma única imagem. Os **Modelos Baseados em Fluxo** oferecem alternativas que repensam fundamentalmente como conectar ruído e dados. Dentro dessa família, duas abordagens se destacam: os **Normalizing Flows**, que constroem transformações invertíveis e estáticas entre distribuições, e os **Rectified Flow Models**, que aprendem fluxos ODE dinâmicos para conectar ruído a dados de forma mais direta.

Modelos Baseados em Fluxo (Flow-based Models)

Suponha que queremos gerar imagens de rostos. Poderíamos começar com ruído puro — uma distribuição Gaussiana em centenas de dimensões — e gradualmente transformá-lo em algo que se pareça com um rosto. A questão central dos modelos baseados em fluxo é: **como construir uma transformação que seja simultaneamente expressiva o suficiente para gerar dados complexos e tratável o suficiente para que possamos calcular probabilidades exatas?**

Diferentemente de VAEs (que maximizam um limite inferior, a ELBO) e GANs (que não modelam densidade alguma), muitos modelos baseados em fluxo permitem o **cálculo exato da verossimilhança** dos dados — uma propriedade fundamental para comparação objetiva entre modelos e para aplicações que exigem densidades precisas,

como compressão e detecção de anomalias.

Intuição: Como uma Transformação Altera a Densidade

Para entender a matemática dos Normalizing Flows, comece com um exemplo unidimensional simples. Considere $Z \sim \mathcal{N}(0, 1)$ e uma transformação linear $X = f(Z) = 3Z + 1$. Como a densidade de Z se transforma na densidade de X ? A resposta é a **fórmula de mudança de variáveis**:

$$p_X(x) = p_Z(f^{-1}(x)) \cdot \left| \frac{d}{dx} f^{-1}(x) \right| = p_Z\left(\frac{x-1}{3}\right) \cdot \frac{1}{3} \quad (30)$$

O termo $1/3$ é o **fator de distorção de volume**: a transformação linear esticou o eixo por um fator de 3, então a densidade precisa ser comprimida por $1/3$ para que a probabilidade total continue integrando a 1. Esse fator é, em 1D, o valor absoluto da derivada — o análogo unidimensional do determinante Jacobiano.

Por que o Determinante Jacobiano Representa Distorção de Volume

Em d dimensões, o Jacobiano $J_f(x)$ é a matriz $d \times d$ das derivadas parciais de f no ponto x . O determinante $|\det J_f(x)|$ mede quanto a transformação f estica (ou comprime) o volume de uma região infinitesimal ao redor de x :

- $|\det J| = 1$: volume preservado (a área não muda — como uma rotação pura).
- $|\det J| > 1$: volume expandido (a densidade diminui naquela região).
- $|\det J| < 1$: volume contraído (a densidade aumenta naquela região).

Teorema 0.3 (Mudança de Variáveis). *Seja $f_\theta : \mathbb{R}^d \rightarrow \mathbb{R}^d$ uma função invertível e diferenciável que mapeia um dado x a uma representação latente $z = f_\theta(x)$. A densidade de x é:*

$$p_X(x) = p_Z(f_\theta(x)) \cdot |\det J_{f_\theta}(x)| \quad (31)$$

Em escala logarítmica (mais estável numericamente):

$$\log p_X(x) = \log p_Z(f_\theta(x)) + \log |\det J_{f_\theta}(x)| \quad (32)$$

O treinamento de um Normalizing Flow maximiza essa log-verossimilhança exata sobre os dados de treinamento: $\mathcal{L}(\theta) = -\mathbb{E}_{x \sim p_{\text{data}}} [\log p_X(x)]$. Isso contrasta diretamente com VAEs, que maximizam um limite inferior (ELBO), e com GANs, que não modelam verossimilhança alguma.

Normalizing Flows

Definição 0.6 (Normalizing Flow). *Um Normalizing Flow é um difeomorfismo $f_\theta : \mathbb{R}^d \rightarrow \mathbb{R}^d$ — uma transformação invertível e diferenciável — que mapeia a distribuição dos dados $p_{data}(x)$ a uma distribuição latente simples $p_Z(z)$, tipicamente $\mathcal{N}(0, I)$. Amostrando do modelo é simples: $x = f_\theta^{-1}(z)$ com $z \sim p_Z$.*

O nome “flow” (fluxo) vem da composição de K transformações: $f_\theta = f_1 \circ f_2 \circ \dots \circ f_K$. Cada f_k deve ser invertível e ter Jacobiano computável. A composição “flui” a distribuição simples através de deformações sucessivas até atingir a distribuição alvo.

O Desafio do Determinante Jacobiano

A fórmula de mudança de variáveis esconde um problema computacional: para uma matriz $d \times d$ genérica, $\det J$ requer $O(d^3)$ operações. Para uma imagem de 64×64 pixels em RGB, $d = 12288$, e $O(d^3) \approx 10^{12}$ — completamente inviável. Precisamos de uma arquitetura em que o cálculo do determinante seja eficiente.

Affine Coupling Layers

A solução canônica para o problema do Jacobiano são as **affine coupling layers**, introduzidas no NICE (Dinh et al., 2014) e popularizadas pelo RealNVP (Dinh et al., 2016).

Definição 0.7 (Affine Coupling Layer). *Uma coupling layer divide a entrada em duas partes $x = [x_a, x_b]$ e aplica:*

$$z_a = x_a \tag{33}$$

$$z_b = s_\theta(x_a) \cdot x_b + t_\theta(x_a) \tag{34}$$

Onde s_θ (scale) e t_θ (translate) são redes neurais arbitrárias que atuam apenas sobre x_a . A inversa é imediata:

$$x_a = z_a \tag{35}$$

$$x_b = (z_b - t_\theta(z_a)) / s_\theta(z_a) \tag{36}$$

Por que o Determinante é Triangular

A chave da eficiência está na estrutura do Jacobiano. Como $z_a = x_a$, a derivada $\partial z_a / \partial x_a = I$ (identidade) e $\partial z_a / \partial x_b = 0$. Como z_b depende de x_a (através de s_θ e t_θ) mas não de x_b , temos $\partial z_b / \partial x_b = \text{diag}(s_\theta(x_a))$. A matriz resultante é triangular inferior por blocos:

$$J = \begin{pmatrix} I & 0 \\ \partial z_b / \partial x_a & \text{diag}(s_\theta(x_a)) \end{pmatrix} \quad (37)$$

O determinante é o produto dos elementos da diagonal: $\det J = \prod_i s_\theta(x_a)_i$, computável em $O(d)$ independentemente da complexidade de s_θ e t_θ . Isso torna o treinamento viável mesmo para espaços de alta dimensão.

Exemplo 0.3 (Coupling Layer Simples). Considere uma coupling layer com entrada $x = [3, 7]$, dividida como $x_a = [3]$, $x_b = [7]$. Suponha que as redes retornem $s_\theta(3) = 2$ e $t_\theta(3) = 1$. A saída é:

$$z_a = x_a = 3 \quad (38)$$

$$z_b = 2 \cdot 7 + 1 = 15 \quad (39)$$

A inversa recupera $x_b = (15 - 1)/2 = 7$. O determinante Jacobiano é $\det J = 2$, indicando que a transformação esticou o eixo x_b por um fator de 2 — a densidade na direção x_b é reduzida pela metade.

Empilhando Camadas: NICE, RealNVP e Glow

Uma única coupling layer tem capacidade limitada — metade das dimensões passa inalterada. A solução é **empilhar** múltiplas camadas, alternando a cada passo quais dimensões são transformadas e quais passam diretamente:

- **NICE** (2014): Usa coupling layers *aditivas* ($s_\theta = 1$, apenas t_θ). O determinante é sempre 1 (volume preservado). Simples, mas limitado em expressividade.
- **RealNVP** (2016): Introduce o fator de escala s_θ , permitindo compressão e expansão de volume. Alterna a divisão das dimensões a cada camada usando padrões de máscara (checkerboard, channel-wise).
- **Glow** (2018): Substitui as permutações fixas por convoluções 1×1 *invertíveis*, que aprendem a misturar canais de forma otimizada. Introduce também `actnorm` (normalização por ativação) para estabilizar o treinamento.

O empilhamento de K camadas produz $f_\theta = f_1 \circ \dots \circ f_K$, cujo log-determinante é a soma dos log-determinantes individuais:

$$\log \det J_{f_\theta}(x) = \sum_{k=1}^K \log \det J_{f_k}(x_k) \quad (40)$$

Vantagens e Limitações

- **Log-verossimilhança exata:** Diferentemente de VAEs (ELBO) e GANs (sem verossimilhança), Normalizing Flows fornecem a densidade exata dos dados, permitindo comparação objetiva entre modelos.
- **Amostragem e inferência bidirecionais:** A invertibilidade permite tanto gerar ($z \rightarrow x$) quanto codificar ($x \rightarrow z$), útil para compressão, interpolação e detecção de anomalias (amostras com baixa verossimilhança são anomalias).
- **Representação latente interpretável:** Manipulações no espaço z (interpolação linear, operações aritméticas) produzem mudanças semanticamente significativas no espaço x .
- **Limitação arquitetural:** A invertibilidade exige que a dimensionalidade seja preservada em cada camada — a arquitetura é severamente restrita comparada a modelos não-invertíveis.
- **Custo computacional:** Cada coupling layer requer duas passagens pela rede (forward + inverse em treinamento), e o empilhamento de dezenas de camadas torna o treinamento caro.
- **Escalabilidade limitada:** Modelos como Glow exigem GPUs de alta memória e não escalam para megapixels tão bem quanto modelos de difusão.

Enquanto Normalizing Flows constroem uma transformação estática e invertível, os Rectified Flow Models adotam uma abordagem dinâmica: aprendem um campo vetorial que define uma ODE conectando ruído a dados. A diferença fundamental é que Normalizing Flows mapeiam cada ponto a um único ponto correspondente (bijection), enquanto Rectified Flows definem uma trajetória contínua no tempo.

Rectified Flow Models

Os Rectified Flow Models (Liu et al., 2022) surgem como uma evolução direta dos Modelos de Difusão, endereçando sua principal limitação prática: o grande número de etapas necessário para amostragem. Enquanto modelos como DDPM tipicamente requerem centenas ou milhares de passos de denoising — cada um exigindo uma avaliação da rede neural — os Rectified Flows aprendem um fluxo ODE determinístico que conecta ruído a dados de forma mais direta.

Por que a Difusão é Lenta?

Em um modelo de difusão, o processo reverso segue uma cadeia estocástica que gradualmente remove ruído ao longo de T passos. A cada passo, o modelo prevê o ruído residual e subtrai uma fração. Essa trajetória é inerentemente tortuosa: o caminho percorrido no espaço de dados é muito mais longo que a distância geométrica entre o ruído e o dado.

A ideia central dos Rectified Flows é **retificar** (endireitar) esse caminho, aprendendo um fluxo determinístico cujas trajetórias são tão retas quanto possível.

Fundamentos Matemáticos

Seja $x_0 \sim p_{\text{data}}(x)$ uma amostra real e $x_1 \sim \mathcal{N}(0, I)$ uma amostra de ruído. Um Rectified Flow modela um campo vetorial $v : \mathbb{R}^d \times [0, 1] \rightarrow \mathbb{R}^d$ que define uma ODE:

$$\frac{dx_t}{dt} = v(x_t, t), \quad t \in [0, 1] \quad (41)$$

Partindo de x_1 em $t = 1$ e integrando a ODE reversamente, obtemos x_0 em $t = 0$. A trajetória x_t é uma interpolação linear entre ruído e dado:

$$x_t = (1 - t)x_0 + tx_1 \quad (42)$$

O campo vetorial ideal deve apontar de x_1 para x_0 , ou seja, $v(x_t, t) = x_0 - x_1$ em cada ponto da trajetória. O modelo aprende a prever exatamente essa direção.

Função de Perda (Flow Matching)

O treinamento minimiza uma perda de correspondência de fluxo (flow matching):

$$\mathcal{L}_{\text{RF}}(\theta) = \mathbb{E}_{t \sim \mathcal{U}[0,1], x_0 \sim p_{\text{data}}, x_1 \sim \mathcal{N}(0, I)} \left[\|v_\theta(x_t, t) - (x_0 - x_1)\|^2 \right] \quad (43)$$

Onde $x_t = (1 - t)x_0 + tx_1$. O modelo aprende a prever a direção $x_0 - x_1$ em cada ponto da trajetória. Note a semelhança com a perda de denoising dos modelos de difusão: enquanto o DDPM prevê o ruído ϵ adicionado, o Rectified Flow prevê a diferença entre dado e ruído. Ambos são equivalentes em espírito, mas o flow matching permite trajetórias mais curtas.

Processo de Retificação (Reflow)

Uma propriedade única dos Rectified Flows é a **retificação iterativa**. Após treinar um campo v_θ inicial, as trajetórias geradas podem ainda ser curvas. O processo de *reflow* as endireita progressivamente:

1. Amostramos pares (x_0, x_1) da distribuição conjunta induzida pelo fluxo atual (simulamos a ODE e registramos os pontos inicial e final).
2. Treinamos um novo campo v'_θ usando esses pares como supervisão direta.
3. Repetimos até que as trajetórias se tornem linhas retas.

Cada iteração de reflow reduz a curvatura das trajetórias. O processo converge para um fluxo cujas trajetórias são tão retas quanto possível, permitindo amostragem em pouquíssimas etapas — até uma única etapa. Essa é a razão pela qual modelos como Stable Diffusion 3 e SDXL Turbo conseguem gerar imagens de alta qualidade em 1–4 passos.

Vantagens e Aplicações

- **Amostragem rápida:** 1–50 passos vs. 1000 do DDPM tradicional. Um passo já produz amostras reconhecíveis; 4–8 passos são suficientes para qualidade comparável a diffusion com 50 passos.
- **Simplicidade:** A perda de flow matching é mais simples que a ELBO dos modelos de difusão — não requer *reweighting terms* ou schedules complexas de ruído. Basta prever $x_0 - x_1$.
- **Determinismo:** A ODE define um mapeamento determinístico entre ruído e dados, permitindo interpolações consistentes (transições suaves entre imagens) e edição controlada.
- **Estado-da-arte:** O Stable Diffusion 3 utiliza Rectified Flow como mecanismo de geração principal, e o fluxo retificado é o paradigma dominante em modelos de código aberto modernos.

Comparação entre os Paradigmas

Propriedade	Normalizing Flow	Rectified Flow	Diffusion
Invertibilidade	Sim (por construção)	Não	Não
NLL exata	Sim	Não (ELBO aproximado)	Não (ELBO)
Passos de amostragem	1	1–50	10–1000
Estabilidade de treino	Alta	Alta	Alta
Flexibilidade arquitetural	Baixa (invertível)	Alta (qualquer arquitetura)	Alta (qualquer arquitetura)

Enquanto Diffusion e Flow-based models transformam ruído em dados através de processos sequenciais, os **Modelos Baseados em Energia** oferecem uma perspectiva fundamentalmente diferente: em vez de aprender um processo de transformação, eles modelam diretamente uma *paisagem de energia* que atribui valores baixos a amostras plausíveis e valores altos a amostras irreais. A geração de dados se torna então um problema de encontrar os vales nessa paisagem.

Modelos Baseados em Energia

Os Modelos Baseados em Energia (Energy-Based Models, EBMs) oferecem uma perspectiva fundamentalmente diferente para modelagem generativa. Em vez de modelar explicitamente uma distribuição de probabilidade (VAEs), treinar um gerador adversarial (GANs) ou modelar um processo de difusão, os EBMs aprendem uma **função de energia** $E_\theta(x) : \mathbb{R}^d \rightarrow \mathbb{R}$ que atribui valores escalares a cada ponto do espaço de dados.

A intuição vem da física: sistemas físicos tendem naturalmente a estados de menor energia. Uma bola em um relevo irregular rola para os vales; analogamente, uma amostra no espaço de dados deve “rolar” para regiões de baixa energia, que correspondem a padrões plausíveis (rostos, objetos, texturas). Regiões de alta energia correspondem a ruído ou configurações irreais.

Intuição: A Paisagem de Energia

Imagine um conjunto de dígitos manuscritos MNIST. Cada imagem 28×28 é um ponto em um espaço de 784 dimensões. A maioria desses pontos — escolhidos aleatoriamente — parece ruído. Apenas uma fração minúscula do espaço corresponde a dígitos reais. Um EBM aprende uma função $E_\theta(x)$ que atribui:

- **Energia baixa** ($E_\theta(x) \ll 0$): para imagens que se parecem com dígitos reais (vales na paisagem).
- **Energia alta** ($E_\theta(x) \gg 0$): para ruído ou imagens irreais (picos na paisagem).

Para **gerar** uma nova amostra, partimos de um ponto qualquer no espaço e seguimos o gradiente descendente da energia até encontrar um vale. Para **classificar**, comparamos as energias atribuídas por diferentes modelos treinados para cada classe. Para **detectar anomalias**, medimos a energia de uma nova amostra: se for alta, provavelmente está fora da distribuição de treinamento.

Formulação Matemática

A conexão entre energia e probabilidade é dada pela **distribuição de Boltzmann** (ou Gibbs):

Teorema 0.4 (Distribuição de Boltzmann). *Um Energy-Based Model define uma distribuição de probabilidade sobre o espaço de dados $x \in \mathbb{R}^d$ através de uma função de energia $E_\theta(x)$:*

$$p_\theta(x) = \frac{\exp(-E_\theta(x))}{Z_\theta}, \quad Z_\theta = \int \exp(-E_\theta(x)) dx \quad (44)$$

Onde:

- $E_\theta(x) : \mathbb{R}^d \rightarrow \mathbb{R}$ é a função de energia, parametrizada por θ (tipicamente uma rede neural). Quanto menor a energia, maior a probabilidade.
- Z_θ é a **constante de normalização** (ou função de partição), que assegura que $\int p_\theta(x) dx = 1$.

O Problema Central: Z_θ é Intratável

O desafio fundamental dos EBMs é que Z_θ é computacionalmente intratável: requer integração sobre todo o espaço de dados. Para imagens 64×64 RGB, isso significa integrar sobre $64 \times 64 \times 3 = 12288$ dimensões. Mesmo métodos numéricos aproximados falham nessa escala.

Isso contrasta diretamente com outros modelos:

- **Normalizing Flows:** calculam $p(x)$ exatamente via mudança de variáveis.
- **Modelos Autoregressivos:** fatoram $p(x) = \prod p(x_i | x_{<i})$, eliminando a necessidade de Z .
- **VAEs e Diffusion:** maximizam um limite inferior (ELBO), evitando Z .

Os EBMs precisam de métodos especiais de treinamento que contornem a intratabilidade de Z .

Treinamento: Como Aprender a Paisagem de Energia

O treinamento de um EBM segue o princípio de máxima verossimilhança: queremos maximizar $\log p_\theta(x)$ para os dados reais. O gradiente da log-verossimilhança tem uma estrutura reveladora:

$$\nabla_\theta \log p_\theta(x) = -\nabla_\theta E_\theta(x) + \mathbb{E}_{x' \sim p_\theta} [\nabla_\theta E_\theta(x')] \quad (45)$$

Este gradiente decompõe o treinamento em duas forças opostas:

- **Fase positiva** ($-\nabla_\theta E_\theta(x)$): **Cava vales** nos pontos onde há dados reais. Para cada imagem de treinamento, atualizamos θ para *reduzir* sua energia.
- **Fase negativa** ($\mathbb{E}_{x' \sim p_\theta} [\nabla_\theta E_\theta(x')]$): **Ergue picos** nas regiões que o modelo atualmente considera plausíveis. Amostramos do modelo e aumentamos a energia dessas amostras.

O equilíbrio entre essas duas forças é análogo a um escultor que cava vales nos locais corretos (dados reais) e ergue o terreno em todos os outros lugares (amostras do modelo).

Divergência Contrastiva

O termo de fase negativa envolve uma esperança sobre p_θ , que é intratável — afinal, não sabemos Z_θ . A **Divergência Contrastiva** (CD), proposta por Hinton (2002), contorna esse problema aproximando a fase negativa com amostras de uma cadeia MCMC curta, iniciada nos próprios dados:

Definição 0.8 (Divergência Contrastiva (CD- k)). *O gradiente da log-verossimilhança é aproximado executando k passos de MCMC a partir de cada amostra real x :*

$$\nabla_\theta \log p_\theta(x) \approx -\nabla_\theta E_\theta(x) + \nabla_\theta E_\theta(\tilde{x}) \quad (46)$$

Onde $\tilde{x}$ é obtido após k passos de Langevin Dynamics iniciados em x . CD- k é enviesado (a cadeia não converge totalmente), mas surpreendentemente eficaz: CD-1 já é suficiente para aprender representações úteis.

Persistent Contrastive Divergence

A CD vanilla tem uma limitação: a cada iteração, as cadeias MCMC recomeçam dos dados, explorando apenas regiões próximas aos dados reais. A **Persistent Contrastive Divergence** (PCD, Tieleman, 2008) melhora isso mantendo cadeias MCMC *persistentes* que evoluem ao longo do treinamento, em vez de serem reiniciadas:

- Inicializamos um conjunto de cadeias de Markov (*fantasmas*) em pontos aleatórios.
- A cada iteração, executamos um passo de Langevin em cada cadeia.
- Usamos as amostras atuais das cadeias como fase negativa.

PCD permite que a fase negativa explore regiões distantes dos dados de forma mais fiel, reduzindo o viés. No entanto, as cadeias podem “escapar” para regiões de energia muito baixa e arbitrariamente distante, exigindo técnicas de estabilização.

Amostragem: Langevin Dynamics

Para gerar amostras de um EBM — e para executar a CD — precisamos de um método para percorrer a paisagem de energia em direção aos vales. O algoritmo padrão é a **Langevin Dynamics**:

Definição 0.9 (Langevin Dynamics). *Dado um passo $\epsilon > 0$ e uma condição inicial x_0 , a Langevin Dynamics atualiza iterativamente:*

$$x_{k+1} = x_k - \frac{\epsilon}{2} \nabla_x E_\theta(x_k) + \sqrt{\epsilon} z_k, \quad z_k \sim \mathcal{N}(0, I) \quad (47)$$

A iteração converge para a distribuição alvo $p_\theta(x)$ quando $k \rightarrow \infty$ e $\epsilon \rightarrow 0$.

A atualização tem dois termos com interpretações claras:

- $-\frac{\epsilon}{2} \nabla_x E_\theta(x_k)$: O **score** $s_\theta(x) = -\nabla_x E_\theta(x)$ aponta na direção de maior decréscimo da energia — a “força” que empurra a amostra para os vales.
- $\sqrt{\epsilon} z_k$: Ruído Gaussiano que permite ao processo explorar o espaço e escapar de mínimos locais, garantindo convergência para p_θ .

Exemplo 0.4 (Langevin em 1D). Considere um EBM 1D com energia $E(x) = \frac{1}{2}(x - 3)^2$. O score é $-\nabla_x E(x) = -(x - 3)$. A Langevin Dynamics converge para $\mathcal{N}(3, \epsilon/2)$ em vez do alvo desejado $\mathcal{N}(3, 1)$ — o ruído $\sqrt{\epsilon}z$ calibra a largura da distribuição.

Intuição: Sem ruído ($\epsilon = 0$), a dinâmica é puro gradiente descendente e converge para o vale mais próximo ($x = 3$), perdendo toda a estocasticidade. Com ruído, as amostras flutuam ao redor do vale, produzindo uma distribuição.

Desafios Práticos da Langevin Dynamics

- **Aquecimento:** São necessárias dezenas a centenas de passos para que a cadeia se aproxime de p_θ , tornando a amostragem lenta.
- **Taxa de aprendizado:** ϵ deve ser cuidadosamente ajustado — muito alto leva a divergência; muito baixo leva a convergência lenta e baixa mobilidade entre modos.
- **Inicialização:** Partir de ruído puro requer mais passos; partir de amostras reais (como na CD) é mais rápido mas introduz viés.

Score Matching: Treinamento sem MCMC

Uma alternativa radical ao treinamento com CD/Langevin é o **Score Matching** (Hyvärinen, 2005), que elimina completamente a necessidade de amostrar da fase negativa.

A Ideia

Em vez de maximizar a verossimilhança (que requer Z_θ), o score matching minimiza a distância entre o *score* do modelo $s_\theta(x) = -\nabla_x E_\theta(x)$ e o *score* dos dados $\nabla_x \log p_{\text{data}}(x)$:

$$\mathcal{L}_{\text{SM}}(\theta) = \mathbb{E}_{x \sim p_{\text{data}}} \left[\frac{1}{2} \|s_\theta(x)\|^2 + \text{Tr}(\nabla_x s_\theta(x)) \right] \quad (48)$$

O *score* é uma função vetorial que aponta na direção de maior crescimento da densidade — em vez de modelar $p(x)$ diretamente, modelamos seu gradiente. Isso elimina Z_θ porque a constante de normalização desaparece na derivada:

$$\nabla_x \log p_\theta(x) = \nabla_x \log \left(\frac{\exp(-E_\theta(x))}{Z_\theta} \right) = -\nabla_x E_\theta(x) \quad (\text{o termo } Z_\theta \text{ é constante em } x) \quad (49)$$

O Problema do Traço e suas Soluções

O termo $\text{Tr}(\nabla_x s_\theta(x))$ na perda de score matching é caro: $O(d^2)$ na forma ingênua, pois requer calcular o Jacobiano completo de s_θ e somar sua diagonal. Para uma imagem 256×256 , $d = 65536$, e $O(d^2)$ é inviável.

Duas abordagens contornam esse problema:

- **Denoising Score Matching** (Vincent, 2011): Corrompe os dados com ruído e aprende a prever o ruído adicionado — exatamente a perca dos modelos de difusão. Isso elimina o traço.
- **Sliced Score Matching** (Song et al., 2019): Projeta o score em direções aleatórias, reduzindo o custo a $O(d)$ por projeção.

Ponte para Modelos de Difusão

O score matching estabelece uma ponte direta entre EBMs e Diffusion Models:

- **EBMs:** Modelam a energia $E_\theta(x)$. O score $s_\theta(x) = -\nabla_x E_\theta(x)$ é derivado da energia.
- **Diffusion:** Modelam o score diretamente $s_\theta(x_t, t)$ em múltiplos níveis de ruído, sem modelar a energia explicitamente.

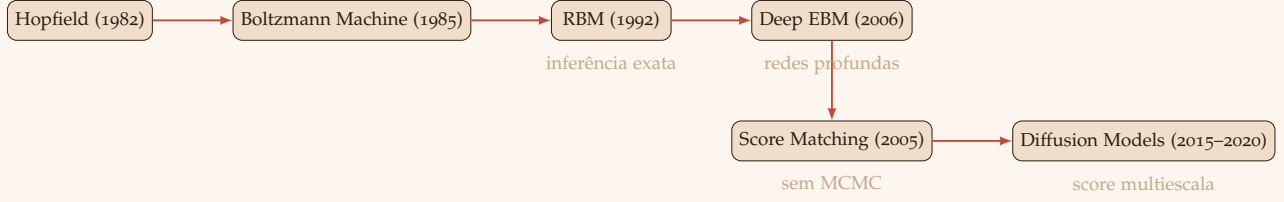
De fato, os modelos de difusão podem ser vistos como EBMs onde a energia depende do nível de ruído t , e o score matching em múltiplas escalas substitui a divergência contrastiva. Essa conexão é a razão pela qual Diffusion Models herdam a estabilidade dos EBMs baseados em score matching.

Síntese: Contexto, Arquiteturas e Relações

Linha do Tempo Histórica

A formulação energética tem raízes profundas no aprendizado de máquina:

- **Redes de Hopfield** (1982): Modelos associativos que minimizam energia $E = -\frac{1}{2} \sum_{i,j} w_{ij} s_i s_j$ para recuperar padrões memorizados.
- **Máquinas de Boltzmann** (Hinton, 1985): Modelos generativos estocásticos com unidades binárias visíveis v e ocultas h , energia $E(v, h) = -v^\top W h - b^\top v - c^\top h$, treinadas com contraste divergente.
- **Restricted Boltzmann Machines** (Smolensky, 1986; Freund & Haussler, 1992): Simplificação das Boltzmann Machines sem conexões visíveis-visíveis ou ocultas-ocultas, permitindo inferência exata.
- **Deep EBMs** (LeCun et al., 2006): Generalização para funções de energia parametrizadas por redes neurais profundas arbitrárias.
- **Score Matching** (Hyvärinen, 2005): Treinamento sem MCMC.
- **Diffusion Models** (Sohl-Dickstein et al., 2015; Song & Ermon, 2019; Ho et al., 2020): EBMs com score matching em múltiplas escalas de ruído.



Treinamento na Prática: Arquiteturas e Estabilização

O treinamento de EBMs profundas é notoriamente instável. A fase negativa (erguer picos) e a fase positiva (cavar vales) competem, e a Langevin Dynamics pode não amostrar p_θ fielmente durante o treinamento. Técnicas práticas incluem:

- **Normalização espectral:** Controla a norma de Lipschitz da função de energia, evitando que a paisagem se torne abrupta e difícil de amostrar.
- **Suavização do gradiente:** Penaliza $\|\nabla_x E_\theta(x)\|$ para evitar picos estreitos que a Langevin não consegue explorar.
- **Buffer de amostras:** Mantém um conjunto de amostras geradas recentemente (replay buffer) para inicializar as cadeias MCMC, reduzindo o custo computacional.
- **Two-stage training:** Pré-treino com score matching ou denoising autoencoder antes de fine-tuning com divergência contrastiva.

Arquiteturas comuns incluem redes convolucionais residuais (ResNet-EBM) e Transformers com cabeça de energia linear. EBMs latentes introduzem variáveis ocultas z com energia conjunta $E_\theta(x, z)$, aumentando a capacidade expressiva ao custo de inferência mais complexa.

Relação com Outros Modelos Generativos

- **VAEs:** Podem ser vistos como EBMs com energia $E(x) = -\log \int p_\theta(x|z)p(z) dz$, mas usando uma distribuição de inferência aproximada $q_\phi(z|x)$ para contornar a intratabilidade. A ELBO é um limite superior na energia.
- **GANs:** O discriminador $D(x)$ pode ser interpretado como uma função de energia: $E(x) = -\log D(x)$. A diferença crucial é que GANs treinam um gerador para produzir amostras (evitando MCMC), enquanto EBMs amostram via Langevin.

Figure 3: Evolução histórica dos modelos baseados em energia, desde as Redes de Hopfield até os Diffusion Models modernos. Cada passo adiciona uma capacidade fundamental: inferência eficiente (RBM), profundidade (Deep EBM), treinamento sem MCMC (Score Matching), e modelagem multi-escala (Diffusion).

- **Diffusion Models:** A conexão mais estreita. Diffusion models aprendem $s_\theta(x_t, t) = -\nabla_{x_t} \log p(x_t) = -\nabla_{x_t} E(x_t, t)$ — o score em cada nível de ruído. São EBMs com score matching multiescala, onde a energia é $E(x, t) = -\log p(x_t)$.
- **Normalizing Flows:** Flows modelam uma transformação invertível explícita; EBMs modelam a densidade implicitamente através da energia. O trade-off é entre tratabilidade (flows podem calcular $p(x)$ exatamente) e flexibilidade (EBMs não têm restrições arquiteturais).

Vantagens e Desvantagens

Vantagens

- **Flexibilidade:** Qualquer função diferenciável $\mathbb{R}^d \rightarrow \mathbb{R}$ pode servir como energia, sem restrições de invertibilidade ou dimensionalidade.
- **Unificação:** VAEs, GANs, RBMs e Diffusion podem ser interpretados como casos particulares de EBMs com diferentes escolhas de energia e treinamento.
- **Detecção de anomalias:** A energia é um detector natural de *out-of-distribution* — amostras fora da distribuição de treinamento tendem a ter energia alta, sem precisar de um classificador separado.
- **Composição:** Funções de energia podem ser somadas para combinar conhecimentos: $E_{\text{total}}(x) = E_1(x) + E_2(x)$ combina restrições de dois modelos.

Desvantagens

- **Amostragem lenta:** Langevin Dynamics requer centenas a milhares de passos para gerar cada amostra, e cada passo exige uma avaliação da rede.
- **Dificuldade de treinamento:** O equilíbrio entre fase positiva e negativa é frágil, e a aproximação MCMC pode levar a instabilidade ou colapso.
- **Qualidade inferior:** Em altas resoluções, a qualidade das amostras geradas por EBMs puros ainda fica atrás de GANs e Diffusion.
- **Cobertura de modos:** A Langevin Dynamics tende a explorar preferencialmente vales profundos, potencialmente ignorando modos de probabilidade significativa mas separados por barreiras de alta energia.

Com os EBMs, completamos nosso panorama dos principais paradigmas generativos. A tabela a seguir sintetiza os compromissos de cada abordagem.

Considerações Finais

Ao longo deste documento, percorremos a evolução dos modelos generativos profundos para síntese de imagens, desde a separação de conteúdo e estilo com CNNs até os Rectified Flow Models e Modelos Baseados em Energia.

Tabela comparativa dos modelos generativos. Cada modelo é avaliado em eixos objetivos: capacidade de calcular densidade exata, número de passos para amostragem, estabilidade de treinamento, flexibilidade arquitetural e aplicações típicas.

Guia de escolha prática.

- **Precisa de densidade exata?** → Normalizing Flow.
- **Precisa de amostragem em tempo real (1 passo)?** → GAN, VAE ou Normalizing Flow.
- **Precisa de máxima qualidade visual?** → Diffusion ou Rectified Flow (com 4–50 passos).
- **Precisa de estabilidade de treinamento?** → VAE, Diffusion, Normalizing Flow ou Rectified Flow.
- **Precisa de espaço latente interpretável?** → VAE ou Normalizing Flow.
- **Precisa de detecção de anomalias sem classificador?** → EBM (energia como score OOD).
- **Precisa de máxima flexibilidade arquitetural?** → Diffusion, EBM ou GAN (sem restrições de invertibilidade).

Direções futuras. A fronteira atual da modelagem generativa move-se em direção a modelos híbridos e unificados:

- **Modelos híbridos:** VAEs com decoders de difusão (VDM), GANs com regularização baseada em energia (EB-GAN), e fluxos retificados condicionados por modelos de difusão para amostragem adaptativa.
- **Consistency Models** (Song et al., 2023): Amostragem em 1–2 passos para modelos de difusão via mapeamento direto ruído-para-dado.

- **Flow Matching** (Lipman et al., 2023): Generalização dos Rectified Flows que simplifica ainda mais o treinamento, substituindo a ODE por uma perda de regressão direta.
- **EBMs neurais modernas**: Combinando score matching com arquiteturas transformer e técnicas de regularização para superar as limitações de escala dos EBM clássicos.
- **Modelos unificados**: Frameworks que tratam diferentes paradigmas (difusão, fluxo, energia) como instâncias de uma mesma formulação matemática, permitindo alternar entre eles conforme a necessidade.